UNIVERSIDAD PERUANA CAYETANO HEREDIA

FACULTAD DE CIENCIAS E INGENIERÍA
CARRERA DE INGENIERÍA INFORMÁTICA



PROYECTO PARCIAL

Programación Orientada a Objetos

Fuga de la Prisión

Diseño e implementación de un videojuego 2D

Curso:	Programación Orientada a Objetos
Docentes:	MSc. Wilder Nina Choquehuayta (principal) MSc. Percy Lovón Ramos (auxiliar) Dr. Ortiz Ortiz Emanuel (auxiliar)
Tema asignado:	Fuga de la Prisión
Ciclo:	IV
Integrantes:	JOSE FERNANDO CHAVEZ LOZANO MICKELE CARLOS TORRES VELAZCO PIERO CEMEI ROMERO MEZA RICHARD AARON SORIANO CORDOVA
Período académico:	2026-II

Lima — Perú

2026

Informe parcial: Fuga de la Prisión

I. Resumen Ejecutivo

Fuga de la Prisión es un videojuego 2D de escape y sigilo con cuatro niveles y un jefe. La arquitectura separa personajes, mapa, ruido, estados, puntaje, combate y dibujo. Sus 24 clases aparecen en el UML y permiten explicar asociación, agregación, composición, encapsulamiento, herencia y polimorfismo. Se aplican State y Observer. Los guardias escuchan pasos, investigan y persiguen al jugador. Las pruebas y las capturas muestran el avance disponible.

II. Introducción

a. Contexto y objetivo

El proyecto evalúa la aplicación de la Programación Orientada a Objetos mediante un videojuego. La temática asignada es Fuga de la Prisión: escapar evitando guardias, cámaras y sistemas de seguridad [1]. La entrega parcial busca demostrar un diseño de clases coherente, sus fundamentos POO y los patrones implementados, acompañados de una interfaz y funciones que se puedan ejecutar [2].

La campaña actual tiene tres zonas de sigilo y una arena contra el Alcaide. Cada zona exige conseguir una llave y alcanzar la salida. Los pasos pueden atraer a las patrullas y la visión inicia una persecución; la captura ocurre por contacto. El jugador puede reducir el ruido y usar las paredes para ocultarse.

b. Tecnologías utilizadas

Tecnología	Uso
Python y Pygame	Reglas, ventana, teclado y dibujo de los personajes stickman. Verificado con Python 3.13.7 y Pygame 2.6.1.
Mermaid.js	Fuente editable del UML e imagen generada desde el mismo modelo.
unittest	Pruebas de juego, paredes, ruido, persecución, campaña y correspondencia entre código y documentos.
Git y GitHub	Registro y publicación de versiones. Se revisa el historial real del repositorio.

III. Diseño de Arquitectura Conceptual y de Clases

a. Diagrama de clases UML

El modelo incluye las 24 clases del juego, conforme a la indicación del docente de representar todas. Cada caja tiene nombre, atributos propios y operaciones. Los atributos que guardan otros objetos señalan su clase: mapa: Mapa, ruidos: SistemaRuido, carrera: HabilidadCarrera y jefe: Alcaide. Los miembros heredados se leen en la base. Las propiedades se muestran como atributos y los constructores se omiten en el UML para reducir su tamaño.

El diagrama completo está en la última página ampliada del informe. modelo_parcial.mmd es la fuente editable y modelo_parcial.png se genera con Mermaid.js. Las clases de pruebas no son componentes del videojuego.

b. Estructura de componentes

Archivo y clases	Responsabilidad
juego.py: Juego	Coordina la campaña, conserva puntos y delega en el estado actual.
estados.py: EstadoJuego, Jugando, Pausa, Derrota, Victoria, NivelCompletado	Definen entrada, actualización, pausa y transición entre niveles.
nivel.py: Nivel, Camara, Llave, PuertaSalida	Reúnen los objetos del nivel y resuelven alertas, captura, premios y salida.
mapa.py: Mapa	Conserva el plano, resuelve las colisiones y busca caminos por puertas y pasillos.
personajes.py: Personaje, Prisionero, Guardia, HabilidadCarrera	Movimiento común, controles del jugador, patrulla, persecución y carrera.
ruido.py: SistemaRuido	Registra a los guardias que escuchan y les notifica los pasos que llegan hasta ellos.
puntuacion.py: Puntaje, ObservadorPuntaje, PanelPuntaje	Validan los puntos y comunican sus cambios mediante Observer.
combate.py: Proyectil, Alcaide, Combate	Disparos, ataques del jefe, impactos, vida e inmunidad.
vista.py: Vista	Dibuja interfaz, mapa, personajes y señales de investigación o persecución.

Lectura de las relaciones

El rombo negro representa composición y queda junto al objeto que crea sus partes; el blanco representa agregación. El triángulo vacío apunta a la clase base. La flecha continua indica una referencia guardada y la discontinua un uso o dependencia. 1, 0..1 y 0..* significan uno, ninguno o uno, y ninguno o varios. + indica acceso público; # señala uso interno por convención de Python.

IV. Aplicación de Fundamentos de POO

a. Asociación, agregación y composición

Asociación. Personaje recibe y guarda su Nivel para consultar sus datos y su mapa. SistemaRuido conserva el Mapa recibido, sin crear otro:

```
self.nivel = nivel
self.mapa = mapa
```

La primera instrucción pertenece a Personaje y la segunda a SistemaRuido.

Agregación. Juego crea Puntaje y lo entrega al Nivel. El objeto se conserva al cambiar de nivel. SistemaRuido también guarda guardias que Nivel ya creó y registró como oyentes:

```
self.nivel = Nivel(self.puntaje, self.indice_nivel)
self.ruidos.registrar(guardia)
```

Composición. Nivel crea su Mapa, SistemaRuido, personajes, cámaras, llave y salida. En la arena crea Combate. Prisionero crea HabilidadCarrera y Combate crea Alcaide y Proyectil:

```
self.mapa = Mapa(PLANOS[indice])
self.ruidos = SistemaRuido(self.mapa)
```

b. Herencia y polimorfismo

Prisionero y Guardia heredan de Personaje. Comparten posición y movimiento, pero cada uno implementa update: el prisionero responde al teclado y emite pasos; el guardia percibe, decide y se mueve por su camino. Nivel utiliza una llamada común:

```
for personaje in self.personajes:
    personaje.update()
```

Juego delega actualizar y manejar_entrada en su EstadoJuego activo. Los cinco estados concretos responden según la situación. PanelPuntaje hereda de ObservadorPuntaje e implementa actualizar_puntaje.

c. Encapsulamiento y abstracción

Puntaje conserva _valor y lo modifica mediante set_valor, sumar y reiniciar; el setter rechaza valores inválidos y notifica el cambio. Personaje consulta sus coordenadas con get_posicion y las cambia con mover, que utiliza las colisiones de Mapa. El guion bajo expresa uso interno, aunque no impide por sí solo el acceso directo en Python.

Personaje, EstadoJuego y ObservadorPuntaje son abstractas: sus clases hijas deben implementar las operaciones indicadas. Esto permite compartir una base y conservar el comportamiento propio de cada objeto.

Patrones de diseño implementados

State. EstadoJuego es la base de Jugando, Pausa, Derrota, Victoria y NivelCompletado. Juego delega en el estado activo. La pausa detiene personajes, búsqueda, persecución, temporizadores y combate. NivelCompletado espera Enter antes de cargar otro mapa.

Observer. Puntaje notifica a PanelPuntaje cuando cambia el valor. SistemaRuido aplica una comunicación semejante: registra oyentes y llama escuchar cuando les llega un paso. El emisor avisa sin resolver la conducta posterior del guardia.

VI. Implementación de Resultados Parciales

Se puede mover con WASD o flechas, mantener Ctrl para avanzar con menos ruido y usar Shift para correr. Los guardias investigan la posición de los pasos que escuchan. Si ven al prisionero o una cámara les avisa, lo persiguen por caminos transitables. Al perderlo de vista buscan en la última posición conocida y luego vuelven a patrullar. Solo el contacto provoca la captura.

Los planos tienen perímetros cerrados, puertas definidas y bloques de cobertura, sin suelo aislado ni un recorrido libre por todo el borde. Mapa recorta el movimiento en la primera pared de cada eje y evita atravesarla con desplazamientos grandes o diagonales. El sonido también utiliza los caminos del mapa.



Figura 1. Un guardia investiga pasos de carrera: la interfaz indica ruido alto y una marca ? señala la investigación. Captura obtenida mediante movimiento real del juego.

Resultados comprobados y uso de Git

El 15 de octubre de 2026 se ejecutaron 47 pruebas automáticas sin fallos. Cubren paredes, diagonales, perímetros, conectividad de los mapas, ruido, investigación, persecución, captura, memoria de la última posición, puntaje, estados, transiciones y combate. También comprueban que el UML y la guía representen las clases y el código actuales. La verificación completó los cuatro niveles con movimiento y disparos, sin desactivar enemigos, y generó doce capturas.

La llave concede 100 puntos, cada salida 500 y el jefe 1000: la campaña termina con 3400. Al perder, R repite el nivel y restaura los puntos de entrada; N comienza desde cero. En la arena, Espacio apunta al jefe e IJKL permite disparar en una dirección. La rúbrica pide mostrar un avance del 25 % con interfaces y funciones [2], pero no da una fórmula para medirlo; aquí se presentan funciones comprobables.

Git. Hasta 5dd0d9d, el historial contiene doce commits: diez con el autor configurado JhosFcL, uno con Simei y uno con shzjeud. Registran planificación, implementación, documentación y aportes posteriores. Estos nombres no identifican por sí solos el trabajo de los cuatro integrantes. El detalle se conserva en docs/evidencias/git_estado.md y cada integrante debe explicar sus aportes reales.

VII. Conclusiones

El avance reúne cuatro niveles, sigilo, ruido de pasos, persecución, búsqueda, captura, llaves, salidas, puntaje y combate. Sus 24 clases aparecen en el UML y están explicadas con su código. Se pueden demostrar los seis fundamentos POO solicitados y los patrones State y Observer.

La principal dificultad fue hacer que movimiento, sonido y persecución respetaran las mismas paredes. Separar Mapa y SistemaRuido de la coordinación de Nivel permitió corregir y probar estas reglas. Los casos de puertas, esquinas y pérdida de visión ayudaron a comprobar el comportamiento. El equipo debe ensayar la partida con teclado y preparar la explicación oral.

VIII. Anexos

Fragmento 1. Movimiento de Personaje resuelto por el mismo mapa:

```
self._x, self._y = self.nivel.mapa.desplazar(
    self.get_posicion(), dx, dy, self.radio)
```

Fragmento 2. Notificación de un paso a un oyente:

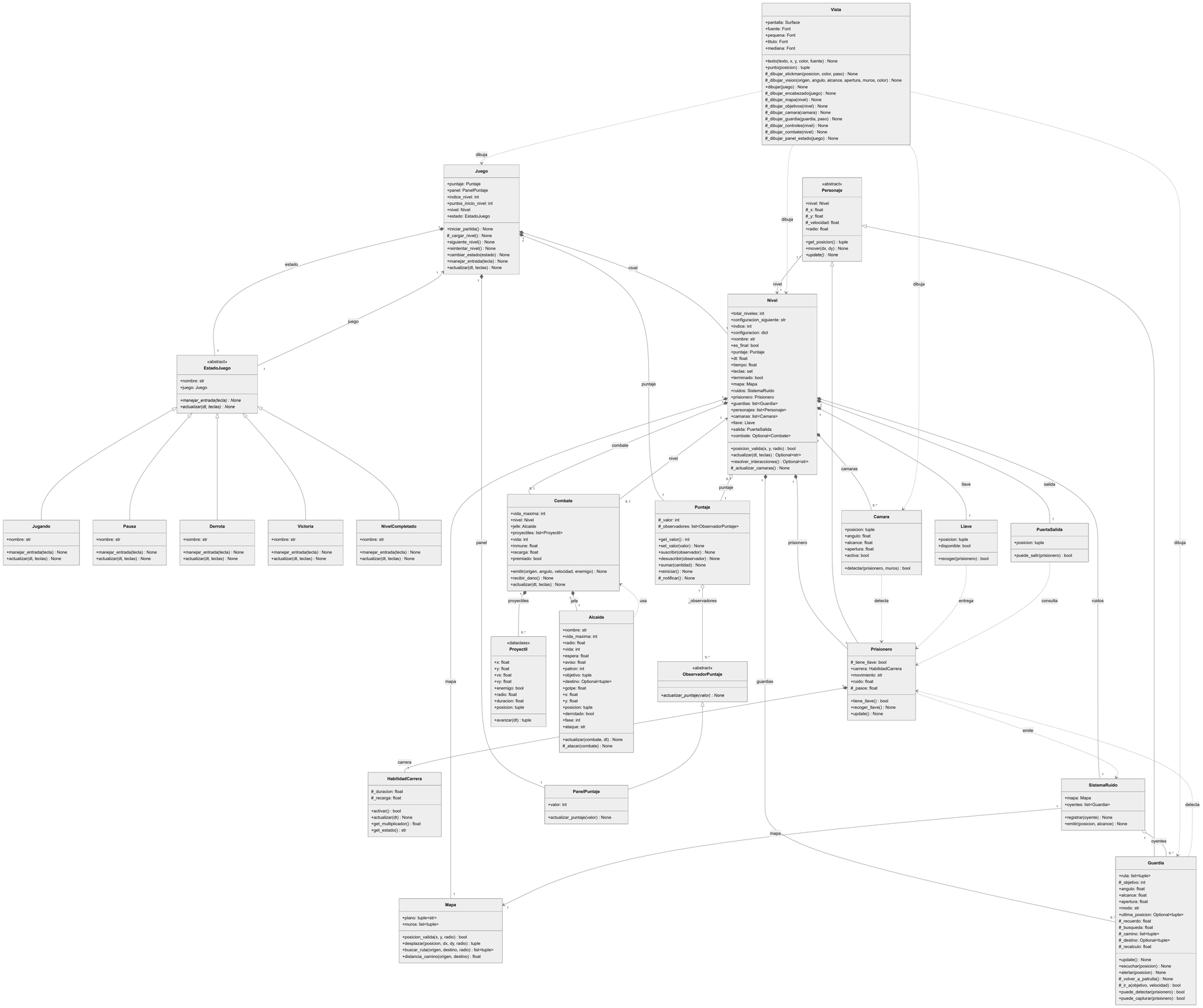
```
recorrido = self.mapa.distancia_camino(
    posicion, oyente.get_posicion())
if recorrido <= alcance:
    oyente.escuchar(posicion)
```

El UML completo está en la página ampliada siguiente. Explicacion_de_clases.md reúne las 24 clases y sus bloques completos. docs/evidencias conserva las capturas y la salida de las pruebas.

Material del curso

[1] Proyecto Final POO, ciclo 2026-2, temática 13.
[2] Rúbrica y Estructura - Proyecto Parcial, ciclo 2026-2.

VIII. Anexos - Diagrama UML completo: 24 clases del juego



Nombre, atributos propios y operaciones. Las propiedades de consulta se muestran como atributos y los miembros heredados se leen en la base.

Rombo negro: composición · Rombo blanco: agregación · Triángulo: herencia · Flecha continua: asociación · Flecha discontinua: dependencia

Fuente: modelo_parcial.mmd · Imagen generada con Mermaid.js · Página 7